Remembrall: Local-First Long-Term Memory for AI Agents

Mahesh Thakur

October 2026

Overview

Remembrall gives AI agents a persistent long-term memory: facts, preferences and commitments accumulate across sessions and are retrievable in single-digit milliseconds. It runs as one local Go binary with one JSON data file - no vector database, no embedding API, no network dependency. The engine combines three ideas that are usually separate systems: an approximate vector index (HNSW), a keyword ranker (BM25), and a memory model with time decay and reinforcement borrowed from how human memory actually behaves.

Why agent memory is not RAG

Retrieval-augmented generation searches static document chunks. Agent memory is stateful in both directions:

- **Writes accumulate** - every session adds memories; the corpus is personal and ever-growing
- **Retrieval mutates state** - a memory that gets recalled is reinforced (harder to forget next time)
- **Time mutates scores** - unused memories decay on an exponential curve
- **Deletion must be total** - “forget this” removes the fact from every index

These properties are the product, and they are what the test suite measures.

Architecture

```
Remember(text) -> Embedder (local, 384-dim) -> HNSW insert
               -> tokenize -> BM25 inverted index
               -> JSON snapshot (atomic rename)

Recall(query) -> HNSW search (ef=200)      -- vector candidates
              -> BM25 search               -- keyword candidates
              -> fusion: score = (0.55*vecSim + 0.45*bm25norm)
                               * decay(days since last access)
                               * (1 + 0.25*importance)
              -> reinforce surfaced memories
```

Local embeddings without an API

Signed feature hashing: token unigrams and bigrams vote into a 384-dimensional vector (dimension from FNV-1a, sign from a second hash bit), L2-normalized. Deterministic, dependency-free, microseconds per document. It captures lexical similarity; the `Embedder` interface is the seam where a small local neural model would plug in for semantic similarity.

HNSW from scratch

The vector index implements Malkov & Yashunin's Hierarchical Navigable Small World (2016): geometric level assignment, greedy descent from the entry point, beam search per layer (efConstruction=200), and the paper's **diverse neighbor-selection heuristic** - a candidate neighbor is kept only if it is closer to the inserted node than to every already-selected neighbor, which keeps graph edges spread across directions instead of stacking one cluster.

BM25 with a real inverted index

Okapi BM25 (k1=1.5, b=0.75) over posting lists, so a query touches only the documents containing its terms. This is what catches exact strings - error codes, names, dates, ticket IDs - that hashing embeddings blur.

Forgetting and reinforcement

Retrieval scores decay exponentially with time since last access (default half-life 30 days). A surfaced memory is reinforced: access count +1, decay clock resets, importance +5% (capped at 1). The test suite verifies the decay constant exactly (120 days at 30-day half-life = 1/16) and the reinforcement side effects.

Measured performance

Method: single goroutine, sequential calls, in-process API, fixed seed, synthetic agent-memory corpus (12 subjects x 16 verbs x 40 objects x 15 details x 50x50 topic pairs - duplicates rare), linux/amd64, Go 1.26.6. Reproduce: `go run ./cmd/bench` (writes `benchmarks.json`), `go run ./cmd/iso` for the index-isolation curve.

Metric	Value
Insert (embed + HNSW + BM25), 50k memories	332 memories/s (150.4s total)
Hybrid recall latency, p50 / p99	4.3 ms / 12.0 ms (500 queries)
HNSW recall@10 vs exact brute force	0.880 (50k embedded corpus, ef=200, 30 queries)
Isolation: adversarial 384-dim gaussian, n=30k	recall@10 0.775 at ef=100, 0.895 at ef=400
Unit test: structured vectors, n=3k	recall@10 = 1.000

What limits recall: an honest account

Three findings, all measured:

1. **Corpus ties destroy the recall metric, not the index.** The first benchmark corpus had ~4k distinct texts for 100k memories, so dozens of vectors were *identical*: exact rank-1 and rank-60 had the same distance. HNSW correctly returned members of the tie cluster while brute force tie-broke differently - recall@10 read 0.21 with both answers correct. The corpus generator was the bug. With a diverse corpus the same code measured far higher.
2. **Uniform random gaussian vectors are the adversarial case** for graph indexes: no manifold structure, all points near-equidistant. Measured isolation curve at n=30k, 384-dim gaussian: recall@10 = 0.775 at ef=100, 0.895 at ef=400. Real embedding distributions are clustered and easier; the unit test's structured vectors give 1.000 at n=3k.
3. **The diverse heuristic costs insert throughput.** Nearest-M linking was ~570 inserts/s at 10k; diverse selection is ~166/s at 30k - a 3.4x insert cost for materially better recall at high dimension. The right tradeoff for a memory system, which writes rarely and reads constantly.

Testing

Eight tests, all passing under `go test -race`:

- embedding determinism and L2 normalization
- related texts score closer than unrelated texts
- HNSW recall@10 >= 0.95 vs exact brute force (n=3,000, structured vectors)
- BM25 ranks exact-term documents first; removal works
- hybrid recall: paraphrase query and exact-term query each surface the right memory
- decay math exact; recall reinforces access count, importance, clock
- forgotten memories never surface again
- save/load round trip preserves recall results

Limitations

- Lexical (not semantic) embeddings; the interface admits a neural drop-in
- HNSW deletion is tombstone-based; periodic rebuild is the production answer
- Two-level fusion weights are hand-set (0.55/0.45), not learned
- Sequential benchmarks by design

Conclusion

A working memory system is a retrieval engine plus a state model. Remembrall implements both from scratch, measures them honestly - including the measurement itself failing (the tie-corpus bug) - and documents the tradeoffs where the literature is optimistic and the data is not.